

AIGC检测 · 全文报告单

NO:CNKIAIGC2026FJ_202605129705306

检测时间: 2026-05-12 20:14:33

篇名: 基于CNN与YuNet的人脸表情识别系统设计与实现

作者: 张吉鹏

单位:

文件名: 1354722_张吉鹏_基于CNN与YuNet的人脸表情识别系统设计与实现.docx

全文检测结果



AI特征值: 30.9%

AI特征字符数: 10198

总字符数: 33051

- AI特征显著 (计入AI特征字符数)
- AI特征疑似 (未计入AI特征字符数)
- 未标识部分

AIGC片段分布图

前部20%

AI特征值: 24.2%

AI特征字符数: 1597

中部60%

AI特征值: 38.4%

AI特征字符数: 7622

后部20%

AI特征值: 14.8%

AI特征字符数: 979



分段检测结果

序号	AI特征值	AI特征字符数 / 章节(部分)字符数	章节(部分)名称
1	37.1%	1597 / 4303	中英文摘要等
2	12.7%	537 / 4236	第1章绪论
3	83.4%	5052 / 6056	第2章系统需求分析与架构设计
4	27.2%	1651 / 6060	第3章表情识别模型构建与训练

5	9.8%	382 / 3910	第4章模型训练优化
6	10.7%	670 / 6265	第5章系统实现与运行测试
7	13.9%	309 / 2221	第6章结论与展望

1. 中英文摘要等			
AI特征值: 37.1%		AI特征字符数 / 章节(部分)字符数: 1597 / 4303	

片段指标列表

序号	片段名称	字符数	AI特征		
1	片段1	1597	显著		37.1%

原文内容

摘要

面向智能人机交互和嵌入式视觉应用中对实时情绪感知的需求，本文设计并实现了一套基于CNN与YuNet的人脸表情识别系统。针对传统表情识别系统在端侧平台部署时容易受到模型计算量、检测速度和运行稳定性限制的问题，系统采用“轻量化人脸检测 + 轻量化表情分类 + 树莓派端部署”的整体方案，完成从视频采集、人脸定位、表情分类到结果显示的完整识别流程。

在模型设计方面，系统前端采用YuNet完成人脸检测与关键点定位，为后续表情识别提供稳定的人脸区域输入；表情分类部分以MobileNetV3为骨干网络，将其分类层调整为7类表情输出，用于识别愤怒、厌恶、恐惧、高兴、中性、悲伤和惊讶等表情类别。训练阶段以FER2013格式数据为基础，结合数据增强、类别均衡、伪标签筛选和多阶段训练等策略对模型进行优化，其中伪标签方法主要作为训练优化手段，用于补充监督信号并改善模型分类效果。

在系统实现方面，本文完成了PyTorch模型训练、ONNX模型导出、TensorFlow SavedModel转换以及TensorFlow Lite模型生成，并将量化后的模型部署到树莓派5平台。树莓派端程序集成摄像头视频采集、YuNet人脸检测、人脸区域裁剪、表情分类推理、结果可视化显示和图像保存等功能。为降低端侧推理负载，系统在部署阶段采用降低检测输入分辨率、间隔执行检测、限制单次分类人脸数和异步视频读取等优化方式，提高了运行过程的连续性和响应速度。

实验与运行测试结果表明，最终表情分类模型在测试集上取得90.77%的准确率和87.43%的宏平均F1值。部署到树莓派平台后，系统能够在单人室内场景下稳定完

成人脸检测与表情识别任务，整体运行帧率约为40~60 FPS，人脸检测耗时控制在10 ms以内，表情分类耗时约为20~60 ms，连续运行10分钟未出现程序崩溃、摄像头中断或界面卡死等异常情况。结果说明，本文设计的系统已经实现了从模型训练到端侧实时运行的完整闭环，具备较好的实时性、稳定性和工程实现完整性。

关键词：人脸表情识别；卷积神经网络；YuNet；MobileNetV3；树莓派；端侧部署

ABSTRACT

To meet the demand for real-time emotion perception in intelligent human-computer interaction and embedded vision applications, this paper designs and implements a facial expression recognition system based on CNN and YuNet. To address the limitations of traditional facial expression recognition systems when deployed on edge platforms, such as high computational complexity, slow detection speed, and insufficient operational stability, the proposed system adopts an integrated scheme of “lightweight face detection + lightweight expression classification + Raspberry Pi-based edge deployment.” It completes the full recognition pipeline, including video acquisition, face localization, expression classification, and result display.

In terms of model design, YuNet is employed as the front-end module for face detection and facial landmark localization, providing stable facial region inputs for subsequent expression recognition. For expression classification, MobileNetV3 is used as the backbone network, with its classification layer modified to output seven facial expression categories, namely anger, disgust, fear, happiness, neutrality, sadness, and surprise. During the training stage, datasets in the FER2013 format are used as the basis, and the model is optimized through data augmentation, class balancing, pseudo-label filtering, and multi-stage training strategies. Among them, the pseudo-labeling method is mainly used as a training optimization technique to supplement supervisory signals and improve classification performance.

In terms of system implementation, this paper completes PyTorch-based model training, ONNX model export, TensorFlow SavedModel conversion, and TensorFlow Lite model generation, and deploys the quantized model on the Raspberry Pi 5 platform.

The program on the Raspberry Pi integrates camera-based video acquisition, YuNet face detection, facial region cropping, expression

37.1%(1597)

classification inference, result visualization, and image saving. To reduce the inference load on the edge device, several optimization strategies are adopted during deployment, including reducing the input resolution for detection, performing detection at intervals, limiting the number of faces classified in a single frame, and using asynchronous video reading, thereby improving the continuity and response speed of the system during operation.

Experimental and runtime test results show that the final facial expression classification model achieves an accuracy of 90.77% and a macro-average F1 score of 87.43% on the test set. After deployment on the Raspberry Pi platform, the system can stably perform face detection and facial expression recognition in single-person indoor scenarios. The overall operating frame rate reaches approximately 40 - 60 FPS, the face detection time is controlled within 10 ms, and the expression classification time is approximately 20 - 60 ms. No abnormalities such as program crashes, camera interruptions, or interface freezes occur during 10 minutes of continuous operation. These results demonstrate that the proposed system realizes a complete closed loop from model training to real-time edge deployment, exhibiting good real-time performance, stability, and engineering implementation completeness.

Key Words: facial expression recognition; convolutional neural network; YuNet; MobileNetV3; Raspberry Pi; edge deployment

2. 第1章绪论

AI特征值: 12.7%

AI特征字符数 / 章节(部分)字符数: 537 / 4236

片段指标列表

序号	片段名称	字符数	AI特征		
2	片段1	537	显著		12.7%

原文内容

第1章绪论

1.1 研究背景与意义

随着人工智能、计算机视觉和嵌入式计算技术的发展，智能系统的功能已经从

简单的信息处理逐渐扩展到对用户状态的感知与反馈。人脸表情是人类表达情绪的重要方式之一，具有直观性强、采集方便和非接触等特点，因此人脸表情识别逐渐成为情感计算和人机交互领域中的重要研究方向。近年来，人脸表情识别已被应用于智能交互、课堂状态分析、驾驶员状态监测、心理健康辅助和服务机器人等场景，其研究价值不仅体现在识别算法本身，也体现在实际系统的部署和运行能力上[1-3]。

人脸表情识别的基本任务是从图像或视频中获取人脸区域，并对其情绪类别进行判断。常见表情类别通常包括愤怒、厌恶、恐惧、高兴、中性、悲伤和惊讶等。FER2013等公开数据集为表情识别模型训练和方法对比提供了基础数据来源，使不同模型能够在较统一的数据标准下进行测试与评价[4]。随着深度学习的发展，卷积神经网络逐渐取代传统人工特征方法，成为人脸表情识别中的主流技术路线。相比传统纹理特征和浅层分类器，卷积神经网络能够从图像中自动提取多层次特征，在复杂表情图像识别中具有更强的表达能力。

但是，人脸表情识别系统在实际应用中并不只需要关注离线测试准确率。对于树莓派、移动终端和边缘计算设备等资源受限平台而言，模型的参数量、计算复杂度、推理速度和运行稳定性同样重要。较复杂的深度模型虽然可能在数据集上取得较高准确率，但在端侧平台上运行时容易出现推理延迟高、画面卡顿、内存占用大和连续运行不稳定等问题。因此，如何在保证基本识别效果的前提下，降低模型计算量并完成端侧部署，是当前人脸表情识别系统设计中需要重点解决的问题[1][12][13]。

在轻量化表情识别研究中，研究者通常从网络结构优化、注意力机制、特征融合和模型压缩等角度提升模型效率。例如，基于轻量化ResNet、MobileNet和Transformer改进的表情识别方法，均尝试在参数量、计算量和识别准确率之间取得平衡[5-8]。其中，MobileNetV3由于具有较好的移动端适应性，适合作为嵌入式视觉任务中的轻量化骨干网络[9]。在人脸检测环节，YuNet作为面向边缘设备设计的轻量级人脸检测模型，能够在较低计算开销下完成人脸定位和关键点检测，为端侧实时表情识别提供了可行的前端检测方案[10]。

基于上述背景，本毕业设计围绕端侧人脸表情识别系统的设计与实现展开研究。系统采用“轻量化人脸检测 + 轻量化表情分类 + 树莓派端部署”的整体方案，前端利用YuNet完成人脸检测与关键点定位，后端基于MobileNetV3构建七类表情分类模型，并将训练后的模型部署到树莓派平台，实现从视频采集、人脸检测、表情分类到结果显示的完整流程。相比单纯进行离线模型训练，本设计更强调模型训练、模型转换、端侧部署和运行测试之间的闭环实现，具有较强的工程实践意义。

1.2 国内外研究现状

人脸表情识别技术经历了由传统人工特征方法向深度学习方法发展的过程。早期方法通常先进行人脸检测和预处理，再提取局部纹理、几何结构或统计特征，最

后结合分类器完成表情判断。这类方法结构较清晰，计算量相对可控，但对光照变化、姿态偏转、遮挡和个体差异较敏感，在复杂自然场景下识别稳定性有限。近年来的综述研究表明，深度学习已经成为人脸表情识别领域的主流方向，研究重点也逐渐从单一模型准确率扩展到数据质量、模型泛化能力、轻量化设计和实际部署等方面[1-3]。

在深度学习方法中，卷积神经网络能够直接从表情图像中学习特征，减少人工设计特征对经验的依赖。FER2013数据集的提出推动了深度表情识别方法的发展，使模型可以围绕七类基本表情进行训练和测试[4]。此后，研究者通过改进卷积网络结构、引入注意力机制、融合多尺度特征等方式提高表情识别效果。赵晓等提出基于注意力机制的ResNet轻量网络，通过分组卷积、倒残差结构和多尺度注意力模块减少模型参数量，并在FER2013和CK+数据集上进行了验证[5]。这类研究说明，轻量化设计不仅可以降低模型复杂度，也可以通过合理的特征增强方法保持模型的识别能力。

MobileNet系列网络是轻量化卷积神经网络中的代表结构之一。左义海等提出NCA-MobileNet方法，通过改进MobileNetV3结构并引入注意力机制，实现了面向人脸表情识别任务的轻量化优化[6]。李艳秋等将轻量型Swin Transformer与多尺度特征融合模块相结合，用于改善模型在表情细节捕捉和实时性方面的表现[7]。Jiang等则围绕改进MobileNetV3的人脸表情识别算法展开研究，说明MobileNetV3在表情识别任务中仍具有进一步优化和应用价值[8]。MobileNetV3原始模型通过硬件感知网络搜索和结构优化，在移动端视觉任务中实现了较好的精度与效率平衡[9]。因此，本文选用MobileNetV3作为表情分类模型的骨干网络，能够较好地契合树莓派端部署对模型规模和推理效率的要求。

在人脸检测方面，检测结果的准确性会直接影响后续表情分类输入质量。传统或较复杂的人脸检测模型虽然能够取得较高检测精度，但在端侧平台上运行时可能带来较高计算开销。YuNet是一种面向边缘设备设计的轻量级人脸检测模型，具有模型小、速度快和便于集成等特点，适合作为实时表情识别系统的前端检测模块[10]。此外，也有研究通过改进S3FD等检测网络提高复杂场景下的人脸检测效果[11]。对于本文所设计的系统而言，检测模块不只需要找到人脸，还需要保证在视频流中连续运行，因此轻量化检测方案更符合树莓派端应用需求。

随着边缘计算和嵌入式AI的发展，人脸表情识别系统的研究逐渐从PC端离线测试转向端侧实时部署。Mohammadi等针对边缘设备上的人脸表情识别进行了实验，比较了CPU、GPU、VPU和TPU等不同硬件平台下的运行表现，说明FER模型的实际应用效果与硬件平台、模型结构和推理优化方式密切相关[12]。Liu等对边缘设备上的高效神经网络进行了综述，指出量化、剪枝和网络结构设计是提高端侧模型运行效率的重要方向[13]。Weng等对神经网络量化方法进行了总结，认为量化可以通过降低数值精度来减少模型复杂度和推理开销[14]。Wei等进一步综述了神经网络量化技术的

发展,说明模型压缩和低精度推理仍是端侧部署中的重要研究内容[15]。

综合国内外研究可以看出,人脸表情识别已经形成较为完整的发展路径:传统方法侧重人工特征提取,深度学习方法提升了模型的特征表达能力,轻量化网络和边缘部署技术则推动表情识别系统从离线实验走向实际应用。但在树莓派等资源受限平台上同时完成人脸检测、表情分类、结果显示和稳定运行,仍需要对模型选型、训练过程、模型转换和端侧推理流程进行综合设计。基于此,本文将研究重点放在YuNet人脸检测、MobileNetV3表情分类和树莓派端部署测试上,力求完成一套能够实际运行的端侧人脸表情识别系统。

1.3 本文主要研究内容

围绕端侧人脸表情识别系统的设计与实现,本文主要开展以下几方面工作。

第一,完成系统需求分析与总体架构设计。结合树莓派端实时视觉识别场景,明确系统需要完成摄像头视频采集、人脸检测、人脸区域裁剪、表情分类、结果显示和图像保存等功能。在此基础上,设计系统整体流程,将系统划分为图像采集模块、人脸检测模块、图像预处理模块、表情分类模块、结果显示模块和数据保存模块,为后续模型训练和系统实现提供结构基础。

第二,完成基于MobileNetV3的表情分类模型构建。以FER2013格式数据为训练和测试基础,对图像数据进行尺寸调整、归一化和数据增强处理。模型以MobileNetV3为骨干网络,并将最终分类层调整为七类表情输出。训练过程中结合类别均衡、标签平滑、学习率调度和早停机制等方法提升训练稳定性。对于伪标签相关方法,本文将其作为训练优化策略使用,不将其作为核心创新点进行强调。

第三,完成YuNet人脸检测与表情分类模型的系统集成。系统前端采用YuNet对摄像头输入图像进行人脸检测,并根据检测框裁剪人脸区域;后端将裁剪后的人脸图像输入MobileNetV3表情分类模型,得到对应表情类别和置信度结果。通过检测与分类模块的组合,实现从视频帧输入到表情识别结果输出的完整处理流程。

第四,完成模型转换与树莓派端部署。训练完成后,将PyTorch模型转换为适合端侧运行的格式,并在树莓派平台上完成推理程序设计。针对端侧平台算力有限的问题,系统在实际部署时采用降低检测输入分辨率、间隔执行人脸检测、限制单次分类人脸数量和异步读取视频帧等方式减少运行负载,提高系统实时性和连续运行能力。

第五,完成模型评价与系统运行测试。通过测试集准确率、宏平均F1值和混淆矩阵等指标评价表情分类模型性能;通过功能测试、实时性测试和稳定性测试评价树莓派端系统运行效果。最终从模型识别效果和端侧运行效果两个角度,验证本文系统是否完成从训练到部署的完整闭环。

1.4 本文结构安排

本文共分为六章,各章内容安排如下。

第1章为绪论。主要介绍人脸表情识别的研究背景与意义,梳理传统方法、深度

12.7%(537)

学习方法、轻量化网络和端侧部署相关研究现状，并说明本文的主要研究内容和结构安排。

第2章为系统需求分析与架构设计。主要分析端侧人脸表情识别系统的应用场景、功能需求和性能需求，设计系统总体架构，并对图像采集、人脸检测、表情分类和结果输出等模块进行划分。

第3章为表情识别模型构建与训练。主要介绍数据集构建与预处理方法，说明MobileNetV3表情分类网络的结构调整过程，并对数据增强、类别均衡和训练配置等内容进行说明。

第4章为模型训练优化与实验结果分析。主要围绕模型训练过程展开，分析基础监督训练、多阶段训练和伪标签辅助优化对模型效果的影响，并通过准确率、宏平均F1值和混淆矩阵等指标评价最终模型性能。

第5章为系统实现、模型部署与运行测试。主要介绍模型导出与转换流程、树莓派端推理程序设计、摄像头实时识别流程集成，以及系统功能、实时性和稳定性测试结果，重点验证系统在端侧平台上的实际运行效果。

第6章为结论与展望。主要总结本文完成的工作，分析系统当前存在的不足，并从模型优化、多场景适应性和端侧部署性能等方面提出后续改进方向。

3. 第2章系统需求分析与架构设计

AI特征值: 83.4% AI特征字符数 / 章节(部分)字符数: 5052 / 6056

片段指标列表

序号	片段名称	字符数	AI特征		
3	片段1	312	显著		5.2%
4	片段2	3982	显著		65.8%
5	片段3	758	显著		12.5%

原文内容

第2章系统需求分析与架构设计

2.1 系统需求分析

本毕业设计面向端侧人脸表情识别应用场景，设计并实现一套能够在树莓派平台上运行的人脸表情识别系统。系统以摄像头采集的视频图像作为输入，通过人脸检测模型定位画面中的人脸区域，再将裁剪后的人脸图像送入表情分类模型，最终在显示界面中输出人脸检测框、表情类别和置信度等结果。与单纯的离线图像分类

程序不同，本系统不仅需要完成模型训练和分类测试，还需要实现从视频采集、实时检测、表情识别到结果显示的完整运行流程。

结合课题目标和实际部署条件，本系统的需求主要包括应用场景需求、功能需求、性能需求和部署需求四个方面。通过对这些需求进行分析，可以明确后续系统架构设计、模型选型和部署优化的基本方向。

2.1.1 应用场景与任务目标

本系统主要面向室内近距离人机交互场景，例如课堂状态观察、智能终端交互演示、情绪识别原型系统验证等。在该类场景中，摄像头通常固定在显示器、桌面或嵌入式设备附近，用户与摄像头之间保持相对稳定的距离，系统需要对画面中的人脸进行检测，并判断其当前表情类别。

本系统的识别对象为人脸图像中的基本表情类别，主要包括愤怒、厌恶、恐惧、高兴、中性、悲伤和惊讶七类。系统运行时，需要能够完成以下基本任务：首先，通过摄像头连续采集图像帧；其次，利用人脸检测模型从视频帧中定位人脸区域；再次，对检测到的人脸图像进行裁剪、缩放和归一化处理；最后，利用表情分类模型输出对应表情类别，并将识别结果显示在实时画面中。

从任务目标上看，本毕业设计并不以提出新的复杂网络结构为主要目标，而是围绕端侧实时表情识别系统展开设计。系统的重点在于将轻量化人脸检测模型、轻量化表情分类模型和树莓派端部署流程结合起来，形成能够实际运行的完整识别闭环。

2.1.2 功能需求分析

根据系统任务目标，本文将人脸表情识别系统的功能需求划分为以下几个方面。

第一，视频采集功能。系统需要调用摄像头获取实时视频流，并将采集到的图像帧传递给后续处理模块。视频采集模块应保证输入图像连续稳定，避免因采集延迟或帧读取异常影响后续检测和识别过程。

第二，人脸检测功能。系统需要在输入视频帧中自动检测人脸位置，输出人脸检测框和关键点信息。由于后续表情识别依赖于裁剪后的人脸区域，因此检测框位置应尽量覆盖完整面部区域，避免出现人脸裁剪不完整或背景区域过多的问题。

第三，图像预处理功能。系统需要根据人脸检测结果裁剪人脸区域，并将其调整为表情分类模型所需的输入尺寸。同时，还需要完成颜色通道转换、像素归一化等处理，使输入图像格式与模型训练阶段保持一致。对于实时视频系统而言，预处理过程应尽量简洁，避免占用过多运行时间。

第四，表情分类功能。系统需要对预处理后的人脸图像进行表情识别，输出七类表情中的一种类别以及对应置信度。该功能是系统的核心部分，直接决定识别结果的准确性。考虑到系统部署在树莓派平台上，分类模型既要具有一定特征提取能

5. 2%(312)

65. 8%(3982)

力，也要控制参数量和推理耗时。

第五，结果显示功能。系统需要在实时画面中绘制人脸检测框，并在检测框附近显示识别出的表情类别和置信度，使用户能够直观看到系统识别结果。显示内容应简洁清晰，便于进行系统演示和运行效果观察。

第六，结果保存与调试功能。系统应具备一定的图像保存或结果记录能力，用于保存部分识别画面、测试结果或调试信息。该功能有助于后续分析系统运行效果，也便于在毕业设计中展示系统实际运行情况。

2.1.3 性能需求分析

除功能完整性外，端侧人脸表情识别系统还需要满足一定的性能要求。由于树莓派平台计算资源有限，系统在模型选择和程序设计时不能只追求识别准确率，还需要综合考虑实时性、资源占用和稳定性。

首先，系统需要具备基本识别准确率。表情分类模型应能够在测试集上取得相对稳定的识别结果，并且在实际摄像头画面中能够对常见表情作出合理判断。由于人脸表情受个体差异、光照、姿态和表情强度影响较大，因此系统评价时不仅应关注整体准确率，也需要结合混淆矩阵和各类别表现进行分析。

其次，系统需要满足实时运行要求。实时性主要体现在视频画面是否连续、检测和分类结果是否能够及时更新。在树莓派端运行时，人脸检测和表情分类都需要占用计算资源，因此系统需要通过轻量化模型和推理流程优化降低单帧处理耗时。对于毕业设计原型系统而言，应保证画面显示较为流畅，识别结果能够随视频输入持续更新。

再次，系统需要控制资源占用。树莓派端的CPU、内存和I/O能力均有限，如果模型过大或推理流程过于复杂，容易导致运行延迟增加甚至程序异常退出。因此，系统需要优先选择参数量较小、计算复杂度较低且便于部署的模型结构，并在程序设计时减少不必要的重复计算。

最后，系统需要具备连续运行稳定性。系统在测试过程中应能够持续完成摄像头读取、人脸检测、表情分类和画面显示，避免出现摄像头中断、界面卡死或程序崩溃等问题。稳定性不仅反映模型推理效率，也反映系统各模块之间的协同效果。

2.2 系统总体架构设计

2.2.1 系统总体架构概述

根据系统需求分析，本文采用分层设计思路构建端侧人脸表情识别系统。系统整体技术路线如图2-1所示，主要包括硬件层、数据层、模型训练与伪标签生成层、模型评估与分析层以及在线推理与应用层。各层之间按照“数据准备—模型训练—模型评价—端侧部署”的顺序逐步展开，最终形成从模型训练到树莓派端实时运行的完整闭环。

图 2-1 端侧人脸表情识别系统总体技术路线图

在硬件层，系统主要包括RTX3060 GPU主机、树莓派5和摄像头设备。其中

，RTX3060 GPU主机主要用于表情分类模型的训练和评估，树莓派5作为端侧部署平台，用于运行人脸检测、表情分类和结果显示程序，摄像头设备用于采集实时视频图像。

在数据层，系统使用标注数据集、无标签数据集和伪标签数据集作为模型训练的数据基础。标注数据集主要用于基础监督训练，无标签数据集用于生成伪标签，伪标签数据集则作为训练优化过程中的辅助数据。通过对不同类型数据的组织和利用，可以为后续模型训练提供较完整的数据支撑。

在模型训练与伪标签生成层，系统主要包括监督训练、伪标签生成和半监督训练三个部分。首先利用标注数据完成基础监督训练，得到初始表情分类模型；随后使用训练后的模型对无标签数据进行推理，筛选高置信度样本并生成伪标签；最后将真实标注样本与筛选后的伪标签样本结合，用于后续训练优化。

在模型评估与分析层，系统通过验证集和测试集对训练后的模型进行评价，主要分析模型的准确率、宏平均F1值和混淆矩阵等指标。该部分用于判断模型是否具备后续部署条件，也为模型训练优化提供依据。

在线推理与应用层，系统将训练并转换后的模型部署到树莓派端运行。该层主要包括图像采集模块、YuNet人脸检测模块和MobileNetV3表情识别模块。摄像头采集实时画面后，系统先利用YuNet定位人脸区域，再将裁剪后的人脸图像输入MobileNetV3模型进行表情分类，最终在显示界面输出检测框、表情类别和置信度等结果。

由图2-1可以看出，本文系统不是单独完成某一个模型训练任务，而是围绕端侧人脸表情识别应用，完成了从数据准备、模型训练、模型评价到树莓派端部署运行的整体设计过程。

2.2.2 系统识别流程设计

在总体技术路线中，在线推理与应用层是系统最终运行的核心部分。为了进一步说明树莓派端程序的运行过程，本文对系统实时识别流程进行设计如图2-2所示。该流程主要包括系统初始化、视频帧读取、人脸检测、人脸裁剪与预处理、表情分类、结果显示与保存等步骤。

图 2-2 系统识别流程图

根据系统识别架构，本文设计的实时识别流程如下。

第一步，系统初始化。程序启动后，加载YuNet人脸检测模型和表情分类模型，同时初始化摄像头和显示窗口。系统需要确认摄像头能够正常读取视频帧，并完成模型输入尺寸、类别标签和运行参数等基础配置。

第二步，读取视频帧。系统从摄像头获取当前图像帧，并将其作为后续检测输入。若读取失败，则提示异常或结束程序。

第三步，人脸检测。系统调用YuNet对当前图像帧进行检测，得到人脸检测框和关键点信息。若当前帧未检测到人脸，则直接显示原始画面或提示未检测到人脸。

第四步，人脸裁剪与预处理。对于检测到的人脸区域，系统根据检测框进行适当扩展和裁剪，再将人脸图像缩放到分类模型要求的输入尺寸，并完成归一化处理。

第五步，表情分类。系统将预处理后的人脸图像输入MobileNetV3分类模型，得到七类表情的预测概率，并选取概率最高的类别作为当前表情识别结果。

第六步，结果显示与保存。系统将人脸检测框、表情类别和置信度绘制到视频画面中，并根据设定保存部分识别结果，用于后续测试分析和毕业设计展示。

2.2.3 系统功能模块划分

结合系统架构与识别流程，本文将端侧人脸表情识别系统划分为五个主要功能模块。

1. 图像采集模块

图像采集模块是系统的数据入口，负责调用摄像头读取实时视频流。该模块需要保证画面输入连续稳定，并将每一帧图像传递给后续检测模块。在树莓派端实际运行时，摄像头读取速度会直接影响整体画面流畅度，因此该模块不仅要实现图像输入，还要尽量减少帧读取过程对主循环的阻塞。

2. 人脸检测模块

人脸检测模块负责从视频帧中定位人脸区域。本文采用YuNet作为前端检测模型，输出人脸检测框和关键点信息。检测模块的结果是后续表情分类的基础，如果检测框偏移或人脸区域不完整，分类模型的识别效果会受到影响。因此，该模块需要在检测速度和检测稳定性之间取得平衡。

3. 图像预处理模块

图像预处理模块负责将检测到的人脸区域转换为分类模型可接受的输入格式。该模块主要包括人脸区域裁剪、尺寸缩放、颜色通道处理和归一化等步骤。由于MobileNetV3模型输入需要固定尺寸，预处理模块需要保证实时图像与训练阶段输入格式保持一致。

4. 表情识别模块

表情识别模块是系统的核心功能模块，负责对预处理后的人脸图像进行分类。本文采用MobileNetV3作为表情分类模型骨干，并将输出类别设置为七类基本表情。该模块输出表情类别和预测置信度，为后续显示和记录提供依据。考虑到系统部署在树莓派平台上，模型结构需要尽量轻量，以降低推理耗时。

5. 结果输出与管理模块

结果输出与管理模块负责将识别结果反馈给用户，主要包括在视频画面中绘制人脸框、显示表情类别和置信度，以及保存识别画面或运行结果等。该模块使系统具备可视化演示能力，也方便后续进行实验记录和运行效果分析。

2.3 关键技术与方法选型

本节主要对系统中涉及的关键技术进行选型说明。第2章只对相关技术进行概括

性说明，重点解释其与系统需求之间的对应关系；具体模型结构、训练配置和实验结果将在第3章和第4章中展开。

2.3.1 人脸检测模型选型

人脸检测是表情识别系统的前置环节，其任务是从视频帧中找到有效人脸区域。检测结果的质量会直接影响后续人脸裁剪和表情分类效果。如果检测框偏移较大，分类模型可能接收到不完整的人脸图像；如果检测耗时过长，系统整体帧率会下降。因此，检测模型需要同时满足检测准确性和运行速度要求。

本文选用YuNet作为系统的人脸检测模型。YuNet是一种轻量级人脸检测模型，适合在资源受限平台上进行实时检测。该模型能够输出人脸检测框和关键点信息，为后续人脸裁剪提供基础。YuNet检测效果示意如图2-3所示。

图 2-3 YuNet人脸检测示意图

在本系统中，YuNet主要用于实时视频帧中的人脸定位。考虑到树莓派端计算资源有限，后续系统部署时可通过降低检测输入分辨率、设置检测间隔和限制单次推理人脸数量等方式减少检测模块的计算负载。相比复杂检测模型，YuNet更符合本文“轻量化检测 + 端侧部署”的系统目标。

2.3.2 表情分类模型选型

表情分类模型需要在识别效果和端侧运行效率之间取得平衡。考虑到本系统部署平台为树莓派，分类网络不宜过大，同时又需要具备较好的特征提取能力。MobileNetV3是一种面向移动端和资源受限设备设计的轻量化卷积神经网络，具有较低的计算复杂度和较好的部署适应性。因此，本文选用MobileNetV3作为表情分类模型的骨干网络。

MobileNetV3主要利用深度可分离卷积、倒残差结构、轻量化注意力模块和逐点卷积等结构，在降低计算量的同时保持特征表达能力。其网络结构示意图如图2-4所示。

图 2-4 MobileNetV3网络结构示意图

在本文系统中，MobileNetV3的作用是对裁剪后的人脸图像进行七类表情分类。由于FER2013格式数据中的人脸图像通常分辨率较低，且类别之间存在一定混淆，分类模型需要具备一定的局部特征提取能力。MobileNetV3能够在相对较小的模型规模下提取有效表情特征，符合本系统对实时性和端侧部署的要求。

2.3.3 训练优化方法选型

表情分类模型的效果不仅受网络结构影响，也与训练数据质量和训练策略有关。对于人脸表情识别任务而言，不同类别样本数量可能不均衡，部分表情之间差异较小，容易造成模型对少数类或相似类识别效果不足。因此，在模型训练阶段需要结合一定的优化方法提升训练稳定性。

结合本文训练任务，系统训练阶段主要从数据增强、类别均衡和训练过程稳定性三个方面进行优化。数据增强用于扩展训练样本变化形式，提高模型对姿态、局

12.5%(758)

部遮挡和轻微图像扰动的适应能力；类别均衡用于减弱样本数量差异对模型训练的影响；学习率调度、标签平滑、早停机制和梯度裁剪等方法则用于提高训练过程的稳定性。

此外，本文在训练过程中使用伪标签筛选作为辅助优化手段。该方法主要用于补充可利用样本信息，改善训练数据利用效率。

2.4 本章小结

本章围绕端侧人脸表情识别系统的需求分析与总体架构设计展开说明。首先，结合室内近距离人机交互场景，明确了系统需要完成视频采集、人脸检测、图像预处理、表情分类、结果显示和数据保存等功能，并从识别准确率、实时性、资源占用和稳定性等方面分析了系统性能需求。

其次，本文采用模块化思想设计系统总体架构，将系统划分为图像采集、人脸检测、图像预处理、表情识别和结果输出与管理等模块，形成从摄像头输入到识别结果显示的完整流程。该架构体现了本文毕业设计的核心思路，即通过轻量化检测模型和轻量化分类模型，在树莓派平台上实现端侧人脸表情识别系统。

最后，本章对YuNet人脸检测模型、MobileNetV3表情分类模型以及训练优化方法的选型思路进行了概括性说明，为后续模型构建、训练实验和系统部署奠定基础。下一章将进一步围绕表情分类模型展开，重点介绍数据集构建、预处理方法、MobileNetV3模型调整和训练策略设计。

4. 第3章表情识别模型构建与训练

AI特征值: 27.2% AI特征字符数 / 章节(部分)字符数: 1651 / 6060

片段指标列表

序号	片段名称	字符数	AI特征		
6	片段1	911	显著		15.0%
7	片段2	335	疑似		5.5%
8	片段3	740	显著		12.2%

原文内容

第3章表情识别模型构建与训练

3.1 数据集构建与预处理

第2章已经从系统层面说明了端侧人脸表情识别系统的总体技术路线，明确了系统前端采用YuNet完成人脸检测，后端采用MobileNetV3完成表情分类。本章在此基

基础上，重点围绕表情分类模型展开，说明训练数据的整理方式、数据集划分方法、输入预处理流程、MobileNetV3分类网络调整以及训练策略设计。

表情识别模型的训练效果与数据规模、类别分布和输入格式密切相关。由于不同表情之间存在较强相似性，同一表情在不同个体、不同光照和不同采集角度下也会表现出较大差异，如果训练数据规模不足或类别分布不均衡，模型容易出现对多数类识别较好、对少数类识别不足的问题。因此，本文在FER2013数据集基础上对样本进行了扩充、合并、清洗和重新划分，并将所有样本统一整理为CSV格式，为后续模型训练提供稳定的数据基础。

3.1.1 数据集构建与扩展

本文表情分类任务以FER2013数据集为基础，并在此基础上补充了扩展样本。识别任务共包含七类基本表情，分别为愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性，对应类别编号为0至6。

为了更直观地展示数据扩充效果，本文将原始FER2013数据集与扩充后的数据集进行对比。原始FER2013数据集中，愤怒类3110张、厌恶类248张、恐惧类819张、高兴类9355张、悲伤类4370张、惊讶类4462张、中性类12905张，总计35269张。经过扩充、合并和清洗后，数据集总量达到410246张。本文将七类表情样本数量及总样本数量进行对比，绘制原始FER2013数据集与扩充后数据集的双条形图，如图3-1所示。

图 3-1 原始FER2013与扩充后数据集类别数量对比

由图3-1可以看出，扩充后各类别样本数量均有明显增加。原始数据中样本数量较少的厌恶类和恐惧类，在扩充后分别增加至17128张和16954张，数据稀缺问题得到一定缓解。高兴类和中性类仍然是样本数量较多的类别，说明扩充后数据集中仍存在类别分布不均衡现象。因此，后续训练过程中仍需要结合类别均衡策略，减少样本数量差异对模型训练的影响。

3.1.2 CSV数据合并与数据集划分

本文训练过程中没有直接采用文件夹图像路径方式进行训练，而是将原始样本和扩充样本统一转换为CSV格式。每条CSV记录包含表情类别标签和图像像素信息。这样处理的优点是不同来源的数据可以采用统一格式进行管理，训练脚本读取时也不需要区分样本来源，能够减少数据加载流程中的复杂性。

在数据整理阶段，本文首先将原始训练、验证和测试数据进行合并，生成统一的merged.csv文件。合并完成后，对数据进行随机打乱，并按照类别分别统计样本数量。随后，在每一类内部进行划分，形成训练集、验证集、测试集和无标签集。最终导出的文件包括train.csv、val.csv、test.csv和unlabeled.csv。

本文采用的划分思路为：首先在每个类别内部随机打乱样本，并预留约20%的样本作为无标签集；随后对剩余有标签样本按照8:1:1比例划分为训练集、验证集和测试集。对于不足以构成完整8:1:1划分的零散样本，则并入无标签集，以保证训练集

、验证集和测试集在各类别上保持严格比例关系。需要说明的是，需要说明的是，无标签集在CSV文件中仍保留原始类别字段，该字段仅用于划分后类别分布检查；在伪标签生成和后续辅助训练过程中，程序不使用该字段作为监督标签，而是将该部分样本按照无标签数据处理。

数据集划分结果如表3-1所示。

表 3-1 扩充后数据集划分结果

数据集样本数量主要用途

训练集 262536 用于基础监督训练

验证集 32817 用于训练过程中的模型选择

测试集 32817 用于最终模型性能评价

无标签集 82076 用于伪标签生成与辅助训练

合计 410246 —

各子集类别分布如表3-2所示。

表 3-2 划分后各子集类别分布

表情类别训练集验证集测试集无标签集

愤怒 18208 2276 2276 5692

厌恶 10960 1370 1370 3428

恐惧 10848 1356 1356 3394

高兴 93120 11640 11640 29106

悲伤 29192 3649 3649 9125

惊讶 22608 2826 2826 7072

中性 77600 9700 9700 24259

合计 262536 32817 32817 82076

从表3-2可以看出，训练集、验证集和测试集在各类别上保持了相近的比例关系，有利于保证训练和评估数据分布的一致性。无标签集则用于后续伪标签生成，为多阶段训练提供额外的数据来源。通过这种划分方式，本文既保留了足够规模的有标签训练数据，又为后续训练优化预留了无标签数据基础。

3.1.3 数据预处理与增强策略

由于MobileNetV3模型要求输入图像具有固定尺寸，本文在训练前需要对CSV中的像素数据进行还原和预处理。训练脚本读取CSV文件后，将像素序列还原为图像，并转换为三通道形式，以适配基于ImageNet预训练权重的MobileNetV3模型。随后，输入图像被统一调整为224×224像素。

在训练阶段，为提高模型对不同姿态、轻微旋转、面部局部遮挡和人脸区域变化的适应能力，本文采用了多种数据增强方法，包括随机水平翻转、随机旋转、随机裁剪缩放、归一化和随机擦除等。其中，随机水平翻转可以增强模型对左右姿态变化的适应能力；随机旋转用于模拟摄像头角度或头部轻微偏转；随机裁剪缩放用

15.0%(911)

于增强模型对人脸边界变化的容忍度；随机擦除用于模拟局部遮挡情况。

验证集和测试集不使用随机增强，只进行尺寸调整和归一化处理。这样可以保证评估阶段输入一致，避免随机增强对评价结果造成影响。

通过上述预处理和增强策略，模型在训练过程中能够接触到更多形式的样本变化，有助于提高模型在实际摄像头输入场景中的适应能力。

3.1.4 类别不平衡处理

虽然本文对数据集进行了扩充，但从图3-1和表3-2可以看出，各类别之间仍然存在明显数量差异。例如，高兴类和中性类样本数量较多，厌恶类和恐惧类样本数量相对较少。如果直接采用普通交叉熵损失进行训练，模型容易偏向样本数量较多的类别，从而影响少数类表情的识别效果。

针对这一问题，本文在训练过程中采用类别均衡策略。训练脚本会根据训练集中各类别样本数量计算类别权重，对样本数量较少的类别给予更高的损失权重，从而提高少数类样本在训练过程中的影响。与此同时，损失函数中还加入标签平滑处理，降低模型对单一类别标签的过度自信，提高训练稳定性。

类别均衡和标签平滑的结合，使模型在面对样本分布不均衡和部分类别边界模糊时，能够获得更加稳定的训练效果。

3.2 模型架构设计

3.2.1 MobileNetV3骨干网络选型

本系统最终需要部署到树莓派平台，因此表情分类模型不能只追求识别准确率，还需要控制模型规模和推理开销。综合端侧运行需求和模型特征提取能力，本文选用MobileNetV3作为表情分类模型的骨干网络。

MobileNetV3是一种面向移动端和资源受限设备设计的轻量化卷积神经网络。相比参数量较大的卷积网络，MobileNetV3在保持一定特征表达能力的同时，具有更低的计算复杂度。在本文系统中，MobileNetV3主要负责对裁剪并预处理后的人脸图像进行特征提取，并输出对应的表情类别。

本文采用MobileNetV3-large作为表情识别骨干网络。相比MobileNetV3-small，MobileNetV3-large具有更强的特征表达能力，更适合用于七类表情分类任务。考虑到后续还会通过模型转换和轻量化部署优化降低端侧推理负担，本文在训练阶段优先选择识别效果更稳定的MobileNetV3-large作为基础模型。

3.2.2 分类层调整与输出设计

原始MobileNetV3主要用于ImageNet图像分类任务，默认输出类别数量为1000类。本文表情识别任务只包含七类基本表情，因此需要对网络最后的分类层进行修改。

具体实现时，本文保留MobileNetV3原有特征提取部分，将分类器最后一层全连接层替换为输出维度为7的新线性层。修改后，模型输出七维分类结果，分别对应愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性七类表情。训练阶段，模型输出结果用

5.5%(335)

于计算分类损失；推理阶段，则根据输出概率选取置信度最高的类别作为最终识别结果。

3.2.3 模型评价指标

为了评价表情分类模型的训练效果，本文主要采用准确率和宏平均F1值作为评价指标。

准确率用于衡量所有样本中被正确分类的比例，可以直观反映模型整体识别效果。但在类别分布不均衡的情况下，仅使用准确率可能会掩盖少数类识别不足的问题。例如，如果模型对样本数量较多的高兴类和中性类识别较好，即使对厌恶类或恐惧类识别一般，总体准确率仍可能较高。

因此，本文同时采用宏平均F1值作为模型选择指标。宏平均F1值会分别计算各类别的F1值，再对各类别结果取平均，更适合评价类别不均衡情况下模型对各类表情的综合识别能力。训练过程中，本文以验证集宏平均F1值作为最优模型保存依据，而不是只依据总体准确率进行模型选择。这样能够使模型在整体识别效果和各类别均衡表现之间取得更合理的平衡。

3.3 训练策略设计

本节主要说明表情分类模型的训练方案设计，重点介绍基础监督训练、伪标签辅助训练和多阶段训练流程。需要说明的是，本节只说明训练思路和参数设置，不展开具体训练结果分析。各阶段训练过程、伪标签生成结果和最终实验结果将在第4章中进一步讨论。

3.3.1 基础监督训练方案

基础监督训练是整个训练流程的起点，其目标是利用有标签训练集训练出一个稳定的初始表情分类模型。该模型一方面需要具备基本的七类表情识别能力，另一方面也会作为后续伪标签生成的教师模型。

在该阶段，模型采用MobileNetV3-large作为骨干网络，并加载ImageNet预训练权重进行初始化。训练数据只使用有标签训练集，不引入无标签数据。输入图像经过训练增强后送入模型，模型输出七类表情预测结果，并通过带类别均衡权重和标签平滑的交叉熵损失进行优化。优化器采用AdamW，学习率采用线性预热与余弦衰减结合的方式，以提高训练初期稳定性和后期收敛效果。

基础监督训练阶段不直接使用无标签数据，主要原因是伪标签质量依赖于教师模型的初始判别能力。如果模型尚未稳定时就引入无标签样本，容易产生较多错误伪标签，影响后续训练。因此，本文首先通过纯监督训练获得初始模型，再利用该模型生成伪标签。

3.3.2 伪标签辅助训练方案

在基础监督训练完成后，本文进一步利用无标签集进行伪标签辅助训练。具体做法是：使用前一阶段训练得到的模型对无标签数据进行预测，筛选其中置信度较高的样本，并将预测结果作为伪标签加入后续训练。

伪标签的作用是补充可利用样本信息，提高训练数据利用效率。由于伪标签来源于模型预测，可能存在一定噪声，因此在联合训练时不能简单地将伪标签样本与真实标注样本完全等同处理。

为降低伪标签噪声对训练过程的影响，本文采用了两种控制方式。第一，在伪标签生成阶段设置置信度阈值，只保留高置信度预测样本；第二，在联合训练阶段根据置信度调整伪标签样本的损失权重，使置信度较高的伪标签样本发挥更大作用，置信度较低的样本影响被削弱。通过这种方式，伪标签数据可以以辅助形式参与训练，同时尽量减少错误标签带来的干扰。

3.3.3 多阶段训练流程设计

结合基础监督训练和伪标签辅助训练，本文设计了多阶段训练流程。整体流程分为三个阶段：Stage 1为基础监督训练，Stage 2为第一轮伪标签辅助训练，Stage 3为第二轮伪标签更新后的联合训练。

多阶段训练流程如表3-3所示。
表 3-3 多阶段训练流程设计
阶段训练数据初始化模型主要目标输出模型
Stage 1 有标签训练集 ImageNet预训练模型建立基础表情分类模型 Stage 1最优模型
Stage 2 有标签训练集 + 第一轮伪标签数据 Stage 1最优模型利用伪标签扩展监督信息 Stage 2最优模型
Stage 3 有标签训练集 + 第二轮伪标签数据 Stage 2最优模型更新伪标签质量并继续优化最终分类模型

在Stage 1中，模型只使用真实标注样本进行训练，获得初始分类能力。在Stage 2中，利用Stage 1模型对无标签集生成第一轮伪标签，并将筛选后的伪标签样本与真实标注样本共同用于训练。在Stage 3中，再使用Stage 2模型重新生成伪标签，并继续进行联合训练。

这种多阶段训练流程的目的，是逐步提高数据利用效率和伪标签质量，而不是构建复杂的新算法。通过先监督训练、再逐步引入高置信度伪标签，模型能够在控制噪声影响的同时进一步优化分类能力。

3.3.4 训练参数设置

为了保证训练过程具有较好的稳定性和一致性，本文在各训练阶段采用统一的基础训练参数。

在训练稳定性方面，本文主要采用以下处理方式。首先，学习率使用线性预热和余弦衰减，避免训练初期学习率过大造成参数波动。其次，采用AdamW优化器和权重衰减，以降低模型过拟合风险。再次，启用自动混合精度训练，以减少显存占用并提高训练效率。最后，使用梯度裁剪限制梯度异常波动，增强训练过程稳定性。

在模型保存方面，训练过程以验证集宏平均F1值作为最优模型判断依据。当验

12.2%(740)

证集宏平均F1值提升时，保存当前模型；如果连续多轮没有提升，则触发早停机制。该策略可以避免训练后期过拟合，同时保证最终保存模型具有较好的综合分类能力。

3.4 本章小结

本章围绕表情分类模型的构建与训练方案进行了说明。首先，介绍了数据集来源、数据扩充效果和CSV格式统一处理方式，并通过表格和条形图对原始FER2013数据集与扩充后数据集进行了对比。扩充后数据集总量达到410246张，并进一步划分为训练集、验证集、测试集和无标签集，为后续模型训练和伪标签辅助训练提供了数据基础。

其次，本章介绍了基于MobileNetV3-large的表情分类模型设计。本文保留MobileNetV3的轻量化特征提取结构，将其最后分类层调整为七维输出，使模型能够适配愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性七类表情识别任务。同时，本文采用准确率和宏平均F1值评价模型效果，并将验证集宏平均F1值作为训练过程中保存最优模型的依据。

最后，本章说明了基础监督训练、伪标签辅助训练和多阶段训练流程。下一章将在本章训练方案基础上，进一步分析各阶段训练过程、伪标签生成情况和最终模型实验结果。

5. 第4章模型训练优化

AI特征值: 9.8%AI特征字符数 / 章节(部分)字符数: 382 / 3910

片段指标列表

序号	片段名称	字符数	AI特征		
9	片段1	382	显著		9.8%

原文内容

第4章模型训练优化

4.1 训练优化思路与总体流程

在第3章完成数据集构建、模型结构调整和训练策略设计后，本章进一步对模型训练过程和模型训练结果进行分析。本文的训练目标并不是单纯追求复杂算法结构，而是在MobileNetV3轻量化模型基础上，通过数据扩充、类别均衡、伪标签辅助训练和多阶段优化等方式，获得能够支撑后续树莓派端部署的表情分类模型。

从整体训练流程来看，本文将模型训练分为三个阶段。第一阶段为基础监督训练，仅使用有标签训练集对MobileNetV3-large模型进行训练，目的是获得一个具备

基本表情分类能力的初始模型。第二阶段在第一阶段模型基础上生成第一轮伪标签，并将有标签样本与伪标签样本联合用于训练，使模型能够利用无标签数据中的有效信息。第三阶段使用第二阶段模型重新生成伪标签，并继续进行联合训练，以进一步稳定模型分类能力。

其作用是补充可利用样本信息，提高训练数据利用效率。模型最终效果仍需通过验证集、测试集和混淆矩阵等结果进行综合评价。为了避免与第3章内容重复，本章不再展开训练参数和数据增强细节，而重点分析多阶段训练结果、最终模型评价结果以及各类别识别表现。

4.2 多阶段训练结果对比

为比较不同训练阶段对模型性能的影响，本文分别对Stage 1、Stage 2和Stage 3阶段保存的最优模型进行统一评估。评估时采用验证集和测试集作为评价数据，并统计准确率和宏平均F1值。其中，准确率反映模型总体分类正确比例，宏平均F1值能够更好地反映类别不均衡场景下模型对各类表情的综合识别能力。

多阶段训练结果如表4-1所示。

表4-1 多阶段模型评价结果对比

训练阶段	训练数据	Val Acc/%	Val Macro-F1/%	Test Acc/%	Test Macro-F1/%
Stage 1	有标签训练集	71.57	64.33	71.85	64.57
Stage 2	有标签训练集 + 第一轮伪标签数据	90.58	87.48	90.82	87.56
Stage 3	有标签训练集 + 第二轮伪标签数据	90.55	87.32	90.77	87.43

由表4-1可以看出，Stage 1阶段仅使用有标签样本训练，模型已经能够完成基本七类表情分类任务，但整体效果仍有较大提升空间。该阶段测试集准确率为71.85%，宏平均F1值为64.57%，说明基础模型虽然具备初步识别能力，但在类别分布不均衡和复杂表情样本上仍存在不足。

Stage 2阶段引入第一轮伪标签数据后，模型性能出现明显提升。测试集准确率由Stage 1的71.85%提升至90.82%，宏平均F1值由64.57%提升至87.56%。这说明在基础监督模型已经具备一定判别能力的前提下，引入高置信度伪标签样本能够有效补充训练信息，使模型学习到更加充分的表情特征。

Stage 3阶段使用Stage 2模型重新生成伪标签后继续训练，其测试集准确率为90.77%，宏平均F1值为87.43%，与Stage 2结果基本接近。该结果说明，经过Stage 2训练后，模型性能已经进入相对稳定区间，第二轮伪标签更新没有带来明显增益，但也没有造成模型性能明显下降。因此，后续系统部署采用Stage 3模型作为最终分类模型，主要是因为该模型来自完整的多阶段训练流程，能够较好地衔接模型训练、评估与部署过程。

从整体结果来看，伪标签辅助训练的主要提升集中在Stage 1到Stage 2之间。Stage 3更多体现为训练流程的进一步完善和模型效果的稳定保持。

4.3 最终模型总体评价

在完成多阶段训练后，本文选取Stage 3阶段输出的最终模型进行验证集和测试集评价。最终模型的总体评价结果如表4-2所示。

表4-2 最终模型总体评价结果

数据集 Loss Accuracy/% Macro-F1/%

验证集 0.4054 90.55 87.32

测试集 0.3970 90.77 87.43

由表4-2可以看出，最终模型在验证集和测试集上的结果较为接近。其中，验证集准确率为90.55%，宏平均F1值为87.32%；测试集准确率为90.77%，宏平均F1值为87.43%。测试集结果略高于验证集，但二者差异很小，说明模型在当前数据划分下没有出现明显过拟合，具备较稳定的泛化表现。

从评价指标角度看，准确率超过90%说明模型能够较好地完成整体七类表情分类任务；宏平均F1值达到87%左右，说明模型并不是单纯依靠高频类别获得较高准确率，而是在各类别上也保持了较均衡的识别能力。考虑到本文最终目标是将模型部署到树莓派端进行实时识别，该模型在准确率、类别均衡表现和轻量化结构之间取得了较好的平衡，能够作为后续端侧系统部署的分类模型基础。

需要注意的是，本文的模型训练目标并不是在公开数据集上追求最高指标，而是获得适合端侧部署的稳定分类模型。因此，在最终模型选择时，除了关注测试集准确率外，还需要结合宏平均F1值、混淆矩阵和各类别表现进行综合判断。

4.4 混淆矩阵与类别识别分析

为了进一步分析最终模型在不同表情类别上的识别情况，本文绘制了测试集混淆矩阵，如图4-1所示。图中纵轴表示真实类别，横轴表示预测类别，类别编号0至6分别对应愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性。

图4-1 测试集混淆矩阵

从图4-1可以看出，大部分样本集中在混淆矩阵的主对角线上，说明模型对各类别均具备较好的识别能力。其中，高兴类和中性类样本数量较多，且主对角线位置数值明显较高，表明模型对这两类表情具有较强的识别能力。高兴类共有11121个样本被正确识别，中性类共有9023个样本被正确识别。

为了进一步量化不同类别的表现，本文统计了测试集上各类别的精确率、召回率和F1值，结果如表4-3所示。

表4-3 测试集各类别识别结果

类别编号表情类别 Precision/% Recall/% F1/%

0 愤怒 87.62 87.08 87.35

1 厌恶 88.04 80.07 83.87

2 恐惧 85.27 78.54 81.77

3 高兴 97.05 95.54 96.29

4 悲伤 83.76 81.72 82.73

5 惊讶 90.05 89.07 89.56

6 中性 88.06 93.02 90.47

由表4-3可以看出，高兴类的识别效果最好，其Precision、Recall和F1值分别为97.05%、95.54%和96.29%。这说明高兴表情的视觉特征较明显，例如嘴角上扬、面部肌肉变化较突出，模型较容易学习到该类表情的判别特征。中性类的F1值为90.47%，召回率达到93.02%，说明模型对中性表情也具有较好的识别能力。惊讶类F1值为89.56%，整体表现同样较稳定。

相比之下，恐惧类、悲伤类和厌恶类的F1值相对较低。其中，恐惧类F1值为81.77%，是七类表情中最低的一类；悲伤类F1值为82.73%，厌恶类F1值为83.87%。这说明这些类别在视觉表现上更容易与其他类别产生混淆。例如，恐惧表情在面部局部特征上可能与惊讶、悲伤或中性存在相似性；悲伤表情在表情强度较弱时容易接近中性；厌恶类样本数量相对较少，且面部表现差异较大，因此识别难度也较高。

9.8%(382)

结合混淆矩阵进一步观察可以发现，部分样本容易被预测为中性类。例如，真实悲伤类中有446个样本被误判为中性，真实高兴类中有353个样本被误判为中性，真实厌恶类中有150个样本被误判为中性，真实愤怒类中也有110个样本被误判为中性。这说明在表情强度较弱或面部特征不明显时，模型倾向于将样本归入中性类别。另一方面，中性类也存在被误判为悲伤和高兴的情况，其中有300个中性样本被预测为悲伤，163个中性样本被预测为高兴。这表明中性类别与弱表情类别之间仍存在一定边界模糊问题。

总体来看，最终模型在七类表情上均取得了可用的识别效果，尤其在高兴、中性和惊讶等类别上表现较好；但对于恐惧、悲伤和厌恶等类别，仍存在一定程度的混淆。该结果符合人脸表情识别任务本身的特点，也为后续模型改进提供了方向。后续可以通过进一步提高少数类样本质量、增强弱表情样本标注一致性、引入更加精细的人脸区域特征或注意力机制等方式，改善易混类别的识别效果。

4.5 本章小结

本章围绕表情分类模型的训练优化过程和实验结果进行了分析。首先，本文对Stage 1、Stage 2和Stage 3三个阶段的训练结果进行了对比。结果表明，仅使用有标签数据进行基础监督训练时，模型能够完成初步分类任务，但整体性能有限；在Stage 2中引入高置信度伪标签后，模型准确率和宏平均F1值均得到明显提升；Stage 3在第二轮伪标签更新后与Stage 2结果基本接近，说明模型性能已经趋于稳定。

其次，本文对最终Stage 3模型进行了总体评价。最终模型在验证集上的准确率为90.55%，宏平均F1值为87.32%；在测试集上的准确率为90.77%，宏平均F1值为87.43%。验证集和测试集结果差异较小，说明模型具备较稳定的分类能力。

最后，通过测试集混淆矩阵和各类别指标分析可以看出，模型对高兴、中性和惊讶等类别识别效果较好，但对恐惧、悲伤和厌恶等类别仍存在一定混淆。总体而言，最终表情分类模型已经能够满足后续树莓派端部署和实时识别系统测试的基本要求。下一章将在本章训练结果基础上，进一步介绍模型转换、树莓派端部署和系统运行测试过程。

6. 第5章系统实现与运行测试

AI特征值: 10.7%

AI特征字符数 / 章节(部分)字符数: 670 / 6265

片段指标列表

序号	片段名称	字符数	AI特征		
10	片段1	336	疑似		5.4%
11	片段2	670	显著		10.7%

原文内容

第5章系统实现与运行测试

5.1 系统实现与运行流程

前几章已经完成了端侧人脸表情识别系统的需求分析、模型构建和训练优化。第4章实验结果表明，最终Stage 3模型在测试集上取得了较稳定的识别效果，因此本章进一步围绕模型转换、树莓派端部署和实际运行测试展开说明。

本文系统最终目标不是停留在PC端离线分类实验，而是将训练完成的表情分类模型部署到树莓派端，结合摄像头输入和YuNet人脸检测模型，实现实时人脸表情识别。系统运行时，摄像头采集视频画面，程序调用YuNet检测人脸区域和关键点，然后根据检测框裁剪人脸图像，将其输入TFLite表情分类模型，最后在显示界面中绘制人脸框、关键点、表情类别、置信度、FPS和耗时信息。

在整体实现上，PC端主要承担模型训练、模型导出和格式转换任务；树莓派端主要承担视频采集、人脸检测、表情分类、结果显示和图像保存任务。通过这种方式，本文完成了从模型训练、模型转换到树莓派端运行验证的完整闭环。本章重点通过模型导出文件、端侧推理参数和实际运行截图说明系统实现过程。

5.2 软件环境与硬件平台配置

5.2.1 硬件平台配置

本文系统涉及PC端训练转换平台和树莓派端部署平台。PC端用于数据处理、模型训练、模型评估和模型格式转换；树莓派端用于部署TFLite模型并完成实时识别测试。

5.4%(336)

硬件平台配置如表5-1所示。

表5-1 硬件平台配置

平台配置项具体信息

PC端训练与转换平台处理器 Intel Core i7-10870H @ 2.20GHz

PC端训练与转换平台内存 32.0 GB

PC端训练与转换平台 GPU NVIDIA RTX3060

端侧部署平台开发板 Raspberry Pi 5

端侧部署平台摄像头 H052-8614

端侧部署平台摄像头采集分辨率 640×480

PC端硬件主要用于完成多阶段模型训练和模型转换任务。树莓派5则作为最终部署平台，用于验证模型在端侧环境下的实际运行效果。由于树莓派平台算力和内存资源有限，部署阶段需要控制模型大小、推理频率和单帧处理负载。

5.2.2 软件环境与工具

本文模型训练阶段主要使用PyTorch完成MobileNetV3表情分类模型训练；模型转换阶段使用ONNX、onnxruntime、onnxsim、onnx2tf和TensorFlow Lite等工具；树莓派端使用OpenCV、YuNet和TFLite运行时完成实时推理。模型导出脚本中设置最终导出模型类别数为7，模型名称为MobileNetV3-large，加载的权重文件为best_model_stage3.pth，导出输入尺寸为224，并采用NCHW输入布局。

软件工具及用途如表5-2所示。

表5-2 软件工具与用途

软件或工具主要用途

Python 训练、模型转换和树莓派端推理程序运行环境

PyTorch 加载和训练MobileNetV3表情分类模型

ONNX 作为PyTorch模型到其他框架之间的中间格式

onnxsim 对ONNX模型进行简化

onnxruntime 对导出的ONNX模型进行快速验证

TensorFlow / onnx2tf 将ONNX模型转换为TensorFlow SavedModel

TensorFlow Lite / tflite-runtime 在树莓派端执行TFLite模型推理

OpenCV 摄像头读取、YuNet检测、图像绘制和窗口显示

5.3 模型导出与端侧部署

5.3.1 模型导出与格式转换

第4章确定Stage 3模型作为最终部署模型。该模型最初以PyTorch权重文件形式保存，不能直接在树莓派端高效运行，因此需要将其转换为适合端侧部署的TFLite模型。

本文模型转换过程包括以下步骤。首先，加载训练得到的best_model_stage3.pth权重文件，并构建输出类别数为7的MobileNetV3-large模型

。其次，将PyTorch模型导出为ONNX格式。再次，使用ONNX简化工具生成简化后的ONNX模型。随后，将简化后的ONNX模型转换为TensorFlow SavedModel格式。最后，使用TensorFlow Lite转换器生成TFLite模型。导出脚本中量化方式设置为FP16，验证样本数量设置为200，用于检查转换后模型输出的一致性。

模型转换后得到的主要文件如表5-3所示。

表5-3	模型导出文件
文件类型	大小作用
model.onnx	ONNX文件 16445 KB PyTorch导出的ONNX中间模型
model_simplified.onnx	ONNX文件 16459 KB 简化后的ONNX模型
saved_model	文件夹 — TensorFlow SavedModel目录
saved_model.pb	PB文件 16570 KB SavedModel模型文件
model_simplified_float32.tflite	TFLite文件 16451 KB float32格式TFLite模型

model_fp16.tflite TFLite文件 8265 KB FP16量化后的TFLite模型

由表5-3可以看出，FP16量化后的 model_fp16.tflite 文件大小约为8265 KB，而float32格式TFLite模型约为16451 KB。FP16量化后模型文件体积约减少一半，能够降低树莓派端存储占用，也更适合端侧部署。

5.3.2 模型转换一致性验证

模型格式转换可能导致输出结果发生偏移，因此在部署前需要进行一致性验证。本文使用转换脚本对PyTorch、ONNX和TFLite三种形式的模型输出进行对比，验证样本数量为200。验证报告显示，PyTorch与ONNX的平均最大绝对误差约为，PyTorch与TFLite的平均最大绝对误差约为0.0068，ONNX与TFLite的平均最大绝对误差也约为0.0068；三种模型在验证样本上的预测一致性均为1.0。

模型转换一致性验证结果如表5-4所示。

表5-4	模型转换一致性验证结果	
对比项	平均最大绝对误差	平均绝对误差
PyTorch 与 ONNX	2.50×10^{-6}	1.18×10^{-6}
PyTorch 与 TFLite	0.0068	0.0033
ONNX 与 TFLite	0.0068	0.0033

从表5-4可以看出，PyTorch模型与ONNX模型之间误差极小，说明ONNX导出过程基本保持了模型输出一致性。TFLite模型由于采用FP16量化，输出与原PyTorch模型之间存在小幅数值差异，但差异范围较小，预测结果保持一致。因此，本文认为该TFLite模型能够作为树莓派端部署模型使用。

5.4 树莓派端推理程序设计

5.4.1 推理程序总体结构

树莓派端推理程序主要包括摄像头读取、YuNet人脸检测、目标跟踪、人脸区域

裁剪、TFLite表情分类、结果绘制和图像保存等部分。程序启动后，首先加载TFLite表情分类模型和YuNet人脸检测模型，然后初始化摄像头并进入实时循环。

树莓派端推理程序加载FP16量化后的model_fp16.tflite作为表情分类模型，并加载face_detection_yunet_2023mar.onnx作为YuNet人脸检测模型。程序中的表情类别标签依次为anger、disgust、fear、happy、sad、surprise和neutral，输入图像尺寸设置为224，并使用与训练阶段一致的ImageNet均值和标准差进行归一化处理。

推理程序主要流程如下：

第一，摄像头读取视频帧。程序通过异步摄像头读取类获取实时图像，避免摄像头读取阻塞主循环。

第二，YuNet检测人脸。程序对缩放后的输入图像执行人脸检测，得到检测框和五点关键点。

第三，目标跟踪与位置更新。对于非检测帧，程序利用上一轮检测结果维持目标状态，减少重复检测开销。

第四，人脸区域裁剪。程序根据检测框扩展人脸区域，并裁剪出分类模型输入图像。

第五，TFLite模型推理。裁剪后的人脸图像经过RGB转换、缩放和归一化后输入TFLite模型，输出七类表情概率。

第六，结果绘制与保存。程序在画面中绘制检测框、关键点、预测类别、置信度、FPS和耗时信息，并异步保存高置信度图像。

5.4.2 树莓派端运行参数

树莓派平台算力有限，如果对每一帧图像都执行完整检测和分类，容易造成画面卡顿。因此，本文在端侧推理程序中设置了多项运行参数，以控制检测和分类负载。

主要运行参数如表5-5所示。

表5-5 树莓派端推理参数设置

参数设置值作用

摄像头分辨率 640×480 获取实时输入画面

处理分辨率 640×480 主程序处理图像尺寸

YuNet检测输入尺寸 256×192 降低人脸检测计算量

检测间隔 detect_every=3 每3帧执行一次完整检测

分类间隔 infer_every=2 避免连续帧重复分类

YuNet置信度阈值 0.7 过滤低置信度检测框

NMS阈值 0.3 去除重复检测框

TFLite线程数 4 提高分类推理效率

表情置信度阈值 0.45 过滤低置信度分类结果

人脸框扩展比例 0.18 保留较完整的人脸区域
最大跟踪人脸数 4 控制画面中最多跟踪目标
单帧最多分类人脸数 1 降低多目标分类负载
目标帧率参数 30 FPS 控制显示刷新节奏

在人脸检测部分，程序将检测输入尺寸设置为 256×192 ，检测结束后再将检测框和关键点映射回原始画面尺寸。这样可以减少YuNet检测阶段的计算量，同时保证画面中检测框位置正确。

在表情分类部分，程序只对主要人脸进行分类推理。虽然程序最多可跟踪4个人脸，但单帧最多只对1个人脸执行表情分类，这一设置能够降低多目标场景下的计算压力。对于毕业设计中的室内单人演示场景，该策略能够在识别功能和实时性之间取得较好的平衡。

5.5 系统运行测试与效果分析

5.5.1 功能测试

为验证系统各模块能否正常运行，本文对摄像头采集、人脸检测、表情分类、结果显示、图像保存和多人场景显示等功能进行了测试。

从功能测试结果看，系统已经能够完成从摄像头输入到识别结果显示的完整流程。YuNet模块能够检测人脸并输出关键点，TFLite分类模型能够输出表情类别和置信度，结果显示模块能够实时叠加运行信息，图像保存模块能够保存部分识别效果较好的画面。

5.5.2 实时性观察

由于本文未单独使用日志系统持续记录每一帧的性能数据，因此本节实时性结果主要来自系统运行界面中显示的FPS、延迟、检测耗时、分类耗时和主循环耗时。测试场景为室内近距离人脸表情识别，显示方式为VNC远程界面。

从运行截图可以观察到，系统在单人场景下FPS通常保持在约47~57之间，部分画面中FPS显示为52.5、53.6、53.3和56.9；在多人场景示例中，FPS约为50.9。YuNet检测耗时多处于约6~14 ms范围内，说明轻量化检测模型和低分辨率检测输入能够有效降低检测开销。表情分类耗时相对检测耗时更高，多数情况下约为18~36 ms，个别画面中达到60 ms以上，说明TFLite表情分类仍是树莓派端推理流程中的主要耗时部分。不同画面中的延迟存在一定波动，主要与摄像头读取、检测与分类推理负载有关。

系统实时性观察结果如表5-6所示。

表5-6 系统实时性观察结果

指标观察结果
摄像头采集分辨率 640×480
目标显示帧率 30 FPS
运行界面FPS 约47~57 FPS

YuNet检测耗时约6~14 ms

表情分类耗时约18~36 ms

测试场景室内近距离单人及少量多人场景

从表5-6可以看出，系统在室内测试环境下能够保持较连续的画面显示，检测框和识别类别能够随人脸状态变化进行更新。YuNet检测耗时较低，主要得益于检测输入尺寸被降低至256×192以及间隔检测策略。分类耗时相对检测耗时更高，说明TFLite表情分类仍然是树莓派端主要计算开销之一。

5.5.3 运行效果展示

为了展示系统实际运行效果，本文选取了若干树莓派端运行截图，覆盖愤怒、恐惧、高兴、中性、悲伤等单人表情识别场景，以及少量多人识别场景。运行效果如图5-1和图5-2所示。

图5-1 单人场景下多类别表情识别效果

图5-1展示了单人场景下的多类别表情识别效果。可以看出，系统能够在画面中绘制人脸检测框和五点关键点，并在检测框附近显示目标编号、预测类别和置信度。画面左上角同时显示FPS、延迟、检测耗时、分类耗时和主循环耗时等信息，便于观察系统实时运行状态。

图5-2 多人场景下表情识别效果

图5-2展示了多人场景下的识别效果。当画面中出现两个人脸时，系统能够检测并绘制多个人脸框。由于程序设置了max_infer_faces=1，系统优先对主要目标进行表情分类，以降低多人场景下的分类推理负载。因此，当前系统更适合单人近距离实时识别场景，在多人场景下仍需要进一步优化多目标分类策略。

5.5.4 稳定性测试

为验证系统连续运行能力，本文在树莓派端对系统进行了稳定性观察。测试过程中，系统持续执行摄像头读取、人脸检测、表情分类、结果显示和图像保存等操作，观察程序是否出现崩溃、摄像头中断、界面卡死或结果无法更新等异常情况。

测试结果表明，在室内近距离场景下，系统能够较稳定地完成实时采集、检测、分类和显示任务。异步摄像头读取和异步图像保存机制对系统稳定性有一定帮助，使视频读取和图像写入不会明显阻塞主循环。

不过，当前稳定性测试主要基于人工观察，测试时长和场景复杂度仍然有限。后续可以进一步增加长时间运行测试和多场景测试，例如弱光、逆光、多人移动、侧脸和遮挡等情况，以更全面地评价系统稳定性。

5.6 本章小结

本章围绕人脸表情识别系统的实现、模型部署和运行测试进行了说明。首先，本文介绍了PC端与树莓派端的分工关系，PC端主要完成模型训练、导出和转换，树莓派端主要完成摄像头输入、YuNet人脸检测、TFLite表情分类和结果显示。

其次，本文说明了模型从PyTorch权重文件转换为ONNX、TensorFlow

10.7%(670)

SavedModel和TFLite模型的过程。最终部署模型采用FP16量化后的model_fp16.tflite，文件大小约为8265 KB，相比float32格式TFLite模型明显减小。模型转换一致性验证结果表明，PyTorch、ONNX和TFLite模型在验证样本上的预测结果保持一致，说明转换后的模型可以用于端侧部署。

最后，本文对树莓派端系统进行了功能测试、实时性观察、运行效果展示和稳定性测试。测试结果表明，系统能够在室内近距离场景下完成摄像头采集、人脸检测、表情分类、结果显示和图像保存等功能，运行画面较为连续，连续运行过程中未观察到程序崩溃、摄像头中断或界面卡死等异常情况。总体来看，本文系统已经实现了从模型训练、格式转换到树莓派端实时运行的完整工程闭环。

7. 第6章结论与展望

AI特征值: 13.9%

AI特征字符数 / 章节(部分)字符数: 309 / 2221

片段指标列表

序号	片段名称	字符数	AI特征		
12	片段1	363	疑似		16.3%
13	片段2	309	显著		13.9%

原文内容

第6章结论与展望

6.1 结论

本毕业设计围绕端侧人脸表情识别系统的设计与实现展开，完成了从数据集构建、模型训练、模型转换到树莓派端实时运行的完整流程。系统以“轻量化人脸检测 + 轻量化表情分类 + 树莓派端部署”为主要思路，前端采用YuNet完成人脸检测与关键点定位，后端采用MobileNetV3-large构建七类表情分类模型，最终在树莓派平台上实现了摄像头采集、人脸检测、表情分类、结果显示和图像保存等功能。

在模型训练方面，本文以FER2013数据集为基础，对样本进行了扩充、合并、清洗和重新划分，并统一转换为CSV格式进行训练。针对表情类别分布不均衡的问题，训练过程中采用了数据增强、类别均衡、标签平滑和伪标签辅助训练等方法。实验结果表明，最终Stage 3模型在测试集上取得了90.77%的准确率和87.43%的宏平均F1值，能够较好地完成愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性七类表情识别任务。其中，高兴、中性和惊讶等类别识别效果较好，恐惧、悲伤和厌恶等类别仍存在一定混淆。

在系统实现方面，本文完成了PyTorch模型到ONNX、TensorFlow SavedModel和

TensorFlow Lite模型的转换，并采用FP16量化方式生成适合端侧运行的TFLite模型。量化后的model_fp16.tflite 文件体积约为8265 KB，相比float32格式模型明显减小。模型转换后，本文还对PyTorch、ONNX和TFLite模型输出进行一致性验证，保证转换后的模型能够用于端侧部署。

在树莓派端部署方面，系统集成了YuNet人脸检测、TFLite表情分类、实时画面显示和异步图像保存等功能。为了降低端侧计算负载，系统设置了较低的人脸检测输入分辨率，并采用间隔检测、间隔分类、限制单帧分类人脸数和异步读取等方式提高运行连续性。实际运行结果表明，系统在室内近距离场景下能够较稳定地完成实时识别任务，运行画面较为连续，检测框、关键点、表情类别和置信度能够正常显示，连续运行过程中未观察到程序崩溃、摄像头中断或界面卡死等异常情况。

总体来看，本文完成了一个可运行的端侧人脸表情识别原型系统。该系统不是单纯停留在离线模型训练层面，而是实现了从模型训练到树莓派端部署运行的工程闭环。

6.2 展望

虽然本文系统已经完成基本功能并能够在树莓派端运行，但仍存在一些不足，需要在后续工作中进一步改进。

第一，模型对部分类别的识别能力仍有提升空间。从实验结果可以看出，恐惧、悲伤和厌恶等类别的F1值相对较低，且容易与中性、惊讶等类别发生混淆。后续可以进一步补充少数类和弱表情样本，提高数据标注一致性，也可以尝试引入更精细的注意力机制或面部局部区域特征，以改善易混类别的识别效果。

第二，系统当前更适合室内近距离单人场景。在多人场景下，程序虽然能够检测多个人脸，但为了保证实时性，当前主要对主要目标进行表情分类，无法同时对所有人脸进行高频分类。后续可以结合更高效的跟踪策略和批量推理方式，提升多人场景下的识别能力。

第三，端侧性能测试还可以进一步完善。本文实时性结果主要来自运行界面观察，虽然能够反映系统基本运行状态，但还缺少长时间自动化日志统计。后续可以在程序中加入性能记录模块，持续统计FPS、检测耗时、分类耗时、CPU占用率和内存占用情况，从而更准确地评估系统在不同场景下的运行表现。

第四，系统还可以进一步优化部署方式。当前模型采用FP16 TFLite格式运行，后续可以继续尝试INT8量化、模型剪枝或硬件加速推理方式，以进一步降低模型体积和推理耗时。同时，也可以针对树莓派端运行环境优化摄像头读取、显示刷新和图像保存逻辑，提高系统整体稳定性和响应速度。

综上，本文系统已经完成端侧人脸表情识别的基本设计与实现，后续可从数据质量、模型结构、多目标识别和端侧性能优化等方面继续改进，使系统在更复杂的实际场景中具有更好的适应能力。

16.3%(363)

致谢

时光匆匆，大学阶段的学习生活即将结束。回顾本次毕业设计从选题、资料查阅、系统设计、模型训练到最终部署测试的整个过程，我在专业知识、实践能力和问题解决能力等方面都得到了较大的锻炼和提升。在此，我谨向所有给予我指导、帮助和支持的老师、同学以及家人表示诚挚的感谢。

首先，衷心感谢我的指导教师康莉莉在毕业设计全过程中给予的耐心指导。从课题方向确定、系统方案设计，到论文结构调整、实验结果分析和格式规范修改，老师都提出了许多宝贵意见，使我能够不断发现问题并及时改进。老师严谨认真的治学态度和负责细致的指导方式，使我受益匪浅。

其次，感谢学院各位老师在本科阶段对我的培养。通过四年的课程学习和实践训练，我逐步掌握了人工智能、计算机视觉、程序设计和系统开发等方面的基础知识，也为本次毕业设计的完成奠定了重要基础。

同时，感谢同学和朋友们在毕业设计过程中给予的帮助与鼓励。在系统调试、模型训练、资料整理和论文修改过程中，他们的交流与建议帮助我更好地理清思路，也让我在遇到困难时能够保持信心。

最后，感谢我的家人一直以来的理解、支持和陪伴。正是他们在学习和生活上的鼓励，使我能够顺利完成本科阶段的学习任务。

本次毕业设计仍有许多不足之处，但它也是我本科阶段一次重要的综合实践。今后我将继续保持学习和探索的态度，不断提升自己的专业能力和实践能力。

附录

13.9%(309)

说明:

- 1、支持中、英文内容检测；
- 2、AI特征值=AI特征字符数/总字符数；
- 3、红色代表AI特征显著部分，计入AI特征字符数；
- 4、棕色代表AI特征疑似部分，未计入AI特征字符数；
- 5、检测结果仅供参考，最终判定是否存在学术不端行为时，需结合人工复核、机构审查以及具体学术政策的综合应用进行审慎判断。



关注微信公众号